Universidad de Zaragoza

Centro Politécnico Superior

Departamento de Informática e Ingeniería de Sistemas

Programación Paralela
Prácticas

Juan Antonio Magallón Lacarta

Índice general

I Memoria Compartida **3**

1. Hilos POSIX **5**

1.1. Introducción 5

1.2. Creación de Hilos 6

1.3. Paso de parámetros 6

1.4. Datos privados. Identificación 7

1.5. Sincronización 8

1.6. Trabajo práctico 9

2. Problema Tipo:
Productor-Consumidor Acotado **11**

2.1. Especificación 11

2.2. Implementación inicial 11

2.3. Acceso exclusivo 11

2.4. Sincronización Productor-Consumidor 12

2.5. Trabajo a presentar 13

2.6. Apéndice 1 14

3. Problema Tipo:
Producto de Matrices
Grafos **17**

3.1. Introducción 17

3.2. Balance de carga 18

II Memoria Distribuida **19**

4. Introducción a MPI **21**

4.1. Configuración de OpenMPI 21
4.1.1. Selección de la implementacion de MPI 21
4.1.2. Configuración de OpenMPI 23
4.1.3. Configuración de OpenSSH 23
4.2. Ejecución de programas en MPI 23
4.3. Trabajo práctico 24

5. Resolución iterativa de sistemas lineales 27

5.1. Resolución iterativa de sistemas de ecuaciones lineales 27
5.2. Implementación sobre MPI 27
5.3. Resultados 28
5.4. Trabajo adicional 28

Introducción

El objetivo de estas prácticas es iniciar al alumno en las herramientas de programación disponibles en Unix para la implementación de algoritmos paralelos. La mayoría de esas herramientas tienen un equivalente en otros sistemas operativos.

Las prácticas se realizarán sobre un sistema de memoria compartida, pero se simulará en alguna de ellas un sistema de memoria distribuida.

Las prácticas se centrarán en algoritmos que requieran una interacción débil entre procesadores, para disminuir la carga de implementación de comunicaciones, haciendo un mayor hincapié en temas más básicos, como la sincronización, el acceso en exclusión mutua a recursos, etc.

El objetivo final es presentar la utilización de las herramientas estándar más conocidas para la implementación de programas paralelos: la biblioteca de *threads* o procesos ligeros definida por el estándar POSIX (posix-threads o pthreads), y la biblioteca MPI. Al estar disponible en prácticamente cualquier sistema Unix permiten programar algoritmos paralelos de forma portable.

Un punto a destacar es que la eficiencia de los algoritmos paralelos en los sistemas extendidos actualmente depende mucho del planificador (*scheduler*) del sistema operativo. Salvo en máquinas específicas o en sistemas COW (**C***lusters* **O***f* **W***orkstations*, Beowulfs), el usuario no puede ligar procesos a procesadores, por lo que la ejecución concurrente de las tareas se basa en el reparto de carga equilibrado por parte de SO entre los procesadores. En la actualidad, sólo se puede disponer de lo que se denomina *processor affinity* en los sistemas SGI y mediante parches para el kernel de Linux: permiten definir grupos de procesadores, aislarlos, ligar procesos a procesadores o grupos (*binding*), etc.

Parte I

Memoria Compartida

Práctica 1

Hilos POSIX

En esta práctica se presenta la utilización básica de lo que se denominan procesos ligeros, hilos de ejecución o *threads*.

1.1. Introducción

En los sistemas operativos actuales existen los conceptos de comunicación entre procesos mediante memoria compartida, que permite que varios procesos compartan datos entre sí para ejecutarse en paralelo. Sin embargo, este método require que todos los datos que se vayan a compartir se creen de una forma especial (por ejemclo con `shmget`). Además los procesos esclavos son eso, procesos completos, por lo que la ejecución de zonas paralelas en partes del programa requiere una sobrecarga excesiva de creación y destrucción de procesos (hay que duplicar los espacios de memoria, los datos del scheduler, etc..).

Los *threads* son la forma más eficiente de paralelización en máquinas de memoria compartida. Son lo que se denominan *procesos ligeros*, es decir, no son procesos independientes sino hilos de ejecución dentro de un proceso que comparten el mismo espacio de direcciones (datos y código) del proceso padre, pero tienen distintos entornos de ejecución. Su creación y destrucción es muy sencilla a nivel del kernel, y al compartir el espacio de direcciones del proceso los datos son compartidos entre todos los hilos de forma automática.

Para crear hilos de ejecución en un programa es necesario utilizar una biblioteca de funciones especiales. Inicialmente, cada sistema que soportaba threads tenía su biblioteca de funciones nativa (`lwp()` en Sun, `sproc()` en SGI, `clone()` en Linux).

El estándar POSIX define una biblioteca de utilización de threads que se denomina *POSIX Threads* o *pthreads*. Ofrece además algunas funciones de sincronización estándar, para simplificar las variaciones entre sistemas a la hora de trabajar con semáforos.

Para utilizar hilos POSIX debemos incluir la cabecera `<pthread.h>` y enlazar con la bliblioteca `libpthread` mediante la opcióm `-lpthread`.

1.2. Creación de Hilos

Las funciones para creación de threads permiten lanzar un hilo en paralelo con el actual que trasfiere el control (punto de ejecución) a una función definida por el usuario. Cuando esa función termina, el hilo termina.

La función de usuario debe ser de la forma:

```
void* slave(void* data)
{
        ...
        return 0;
}
```

El parámetro `data` es un puntero genérico a una zona de datos que el programa padre puede enviar al hilo como datos de llamada. Los hilos se identifican mediante un dato de tipo `pthread_t`.

Para crear el hilo se utiliza la función:

```
int pthread_create(
        pthread_t *thread,
        const pthread_attr_t *attr,
        void *(*start_routine)(void *arg),
        void *arg)
```

El parámetro `attr` permite especificar ciertos atributos especiales para el hilo, que de momento no utilizaremos (se puede enviar un NULL o un 0). Por ejemplo:

```
pthread_t taskID;
pthread_create(&taskID,0,slave,0);
```

Esta llamada crea un hilo, deja su identificador en `taskID`, y llama a la función slave con el valor 0.

Para esperar a que terminen los hilos lanzados, se utiliza la llamada `pthread_join()`:

```
pthread_join(taskID,0);
```

Un hilo termina cuando se sale de la función llamada al crearlo (bien de forma automática o mediante un return), o si en cualquier parte del código que ejecuta el hilo (dentro de esa función o en otra utilizada por ella) se llama a `pthread_exit(0)`.

1.3. Paso de parámetros

El argumento de la función de control del hilo permite enviar parámetros distintos a cada hilo lanzado. El parámtro es de tipo `void *`, por lo que se puede mandar un puntero a un registro con datos. Esos datos deben ser persistentes durante la duración del thread.

Por ejemplo, para el caso de la suma del vector en paralelo, los datos pueden ser el rango que debe sumar cada hilo:

```
typedef struct thrdata_t
{
     int start;
     int end;
} thrdata_t;

void* slave(thrdata_t* info); /* OJO con el prototipo */

thrdata_t info[4];
pthread_t task[4];

int  slice;
slice = numdatos/4;

for (i=0; i<4; i++)
{
     info[i].start = i*slice;
     info[i].end = (i+1)*slice;
     /* Cast para callar al compilador. . . */
     pthread_create(&task[i],0,(void *(*)(void*))slave,&info[i]);
}

for (i=0; i<4; i++)
     pthread_join(task[i],0);
```

1.4. Datos privados. Identificación

En un entorno de ejecución de threads todos los datos son compartidos. Si queremos que exista una variable global a un thread, pero diferente para cada hilo, hay que hacerlo de una forma especial. Vamos a tomar como ejemplo la auto-identificación del thread. Existe una función de la biblioteca de threads llamada `pthread_self()`, pero el valor devuelto es el identificador del thread, que es un dato opaco y no se puede utilizar para nuestros propósitos (identificar cada hilo con un entero de 0 a nproc-1).

Algunos compiladores pueden permitir un modificador similar al static:

```
__thread int self;
for (i=0; i<4; i++)
{
    self = i;
    pthread_create(&task[i],0,slave,&info[i]);
}
void* slave(thrdata_t* info)
{
    print("Soy %d\n",self);
    return 0;
}
```

pero no es estándar ni está soportado en la mayoría de los compiladores.

Podemos mandar el identificador en el argumento del thread:

```
typedef struct thrdata_t
{
    int self;
    ...
} thrdata_t;
for (i=0; i<4; i++)
{
    info[i].self = i;
    pthread_create(&task[i],0,slave,&info[i]);
}
void* slave(thrdata_t* info)
{
    print("Soy %d\n",info->self);
    return 0;
}
```

pero no es visible a otras funciones llamadas desde `slave` (deberiamos estar pasando a todas las funciones que lo necesitaran el valor `info`).

La biblioteca de threads POSIX define un método para crear datos accesibles mediante un único nombre, pero de valor independiente para cada hilo. Para ello se utiliza lo que denominan *claves* o *keys*.

Una clave es el identificador de un dato compartido (que debe ser un puntero); se inicializa con:

```
pthread_key_t self_key;
pthread_key_create(&self_key,0);
```

y se destruye con:

```
pthread_key_delete(self_key);
```

Un hilo puede fijar su valor con:

```
pthread_setspecific(self_key,(void *)info->self);
```

y consultarlo con:

```
int me = (int)pthread_getspecific(self_key);
```

O si queremos ver toda la información:

```
pthread_setspecific(info_key,info);
...
thrdata t* info = (thrdata t*)pthread_getspecific(info_key);
```

NOTA: las claves suelen ser creadas por el proceso padre, y los hilos simplemente fijan un valor individual al comenzar y luego acceden a él.

1.5. Sincronización

La sincronización entre threads puede realizarse con cualquiera de los métodos del sistema operativo, o bien utilizar los candados POSIX estándar.

Un candado de exclusión mutua o *mutex* de la librería de threads es similar a un dato global de thread. Es básicamente un semáforo con un sólo "hueco" de paso.

Un mutex se define mediante un tipo de dato `pthread_mutex_t`, y se inicializa con:

```
pthread_mutex_t mutex;
pthread_mutex_init(&mutex,0);
```

o

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

y se destruye con:

```
pthread_mutex_destroy(&mutex);
```

La zona crítica que requiere acceso en exclusión mutua se protege con el mutex (por ejemplo, un acceso de escritura a una variable global):

```
pthread_mutex_lock(&mutex);
...(zona no accesible en paralelo)...
pthread_mutex_unlock(&mutex);
```

1.6. Trabajo práctico

El programa de partida para la práctica define un vector de reales y calcula la suma de trozos iguales del vector en paralelo. Cada hilo escribe por el terminal su suma parcial.

Tarea 1

Modifica el programa anterior para que los hilos devuelvan los resultados en zonas de memoria compartida pero independientes. No necesitas sincronización.

El hilo principal debe escribir los resultados obtenidos y el total.

Tarea 2

Modifica el programa anterior para que los hilos acumulen los resultados en en una zona de memoria compartida única. Debes proteger la el acceso a la zona compartida con un mutex.

El hilo principal debe escribir el total obtenido.

Tarea 3

Los threads pueden devolver directamente un resultado al hilo principal de control. Consulta las páginas de manual y modifica el programa para que devuelva los resultados directamente como valor de retorno de `slave()` mediante `pthread_exit()`.

OJO: recuerda que los datos devueltos deben ser persistentes, no valen variables locales en `slave()`...

Práctica 2

Problema Tipo: Productor-Consumidor Acotado

En esta práctica se propone la utilización de *threads* para la implementación de un problema típico del modelo productor–consumidor múltiple, el caso de un servidor de impresión.

2.1. Especificación

Se quiere implementar un servidor de impresión multi-hilo que permita que varios hilos clientes envíen trabajos a una cola, que serán recogidos por varios hilos servidores que los imprimen.

2.2. Implementación inicial

Utiliza la implementación básica de una cola circular para implementar el modelo 1-1 (1 cliente, 1 servidor) (ver el Apéndice 1). En el ejemplo se utilizan retardos en el cliente y el servidor para poder simular distintas situaciones de carga.

Podrás observar que el resultado depende mucho del planificador del sistema, del orden en que se lanzen los procesos cliente y servidor, y de los tiempos de espera de cada uno. En la mayoría de los casos se producen errores (cliente que intenta imprimir con la cola llena, servidor que encuentra una cola vacía). El sistema sólo funciona correctamente (la cola no se llena, el servidor no encuentra la cola vacía) en situaciones casuales en las que el *scheduler* planifica los hilos de la manera adecuada.

Además, la implementación de la cola no es segura de cara al multiproceso, puesto que si se lanzan varios clientes, intentarán acceder de forma concurrente a la cola. Lo mismo sucede con los servidores.

2.3. Acceso exclusivo

Modifica la implementación de la cola para que sea segura de cara al acceso concurrente de varios clientes. Para ello, define un mutex en la cola para el acceso de escritura a la cola y protege la funcón *spool_put()*.

Realiza el mismo proceso con el acceso a la cola para lectura en la función *spool_get()*.

Puedes utilizar un bloqueo común ? Es eficiente ?

2.4. Sincronización Productor-Consumidor

A pesar de los procesos de sincronización anteriores, quedan otros problemas pendientes:

- Un cliente no debe mandar un trabajo si la cola está llena.
- Un servidor no debe imprimir si la cola está vacía.

Comenzemos por el primer caso. Un cliente debe comprobar si la cola está llena, y si lo está esperar un hueco:

```
while (s.full) { do_nothing };
put(s,job)
```

Pero ese tipo de bucles utilizan CPU para no hacer nada. Una solución un poco más eficiente podría ser:

```
while (s.full) usleep(10 ms);
put(s,job)
```

Aparece un problema: dos clientes simultáneos pueden estar esperando la cola, y ejecutar su código de forma errónea:

```
Cliente A:                          Cliente B:
while (s.full) usleep(10 ms);
                                    while (s.full) usleep(10 ms);
put(s,job)
                                    put(s,job)
```

Para ello es necesario proteger la zona de espera con un mutex:

```
lock_for_put(s)
    while (s.full) usleep(10 ms);
    put(s,job)
unlock_for_put(s)
```

pero si la varible está protegida con un mutex, no puede cambiar (*deadlock*!!);

Lo ideal sería disponer de algo que nos avisara del cambio en una variable:

```
lock_for_put(s)
    while (s.full)
        unlock_for_put(s)
            wait_for_change(s.full)
        lock_for_put(s)
put(s,job)
unlock_for_put(s)
```

Aun asi, tenemos otro problema: la secuencia de instrucciones:

```
            wait_for_change(s.full)
        lock_for_put(s)
```

debe realizarse de forma atómica, para que otro proceso no se interponga ente el aviso de hueco y la adquisición del mutex.

Para solucionar esto existen en la biblioteca de threads POSIX lo que se denominan *variables de condición*, y las sentencias *pthread_cond_wait()* y *pthread_cond_signal()*.

El código que debe esperar el cambio en la variable de condición tiene la forma:

```
int             ready;
pthread_cond_t  ready_cond;
pthread_mutex_t ready_mutex;

pthread_mutex_lock(&ready_mutex);
    while (!ready)
        // Libera el mutex, espera que se se~nale un cambio en
        // cond_var, y cuando se produce cierra el mutex
        pthread_cond_wait(&ready_cond,&ready_mutex);
    // ready es 1
pthread_mutex_unlock(&ready_mutex);
```

El código que señala el cambio es el siguiente:

```
pthread_mutex_lock(&ready_mutex);
        ready = 1;
    // Marca la variable de condicion
    pthread_cond_signal(&ready_cond);
pthread_mutex_unlock(&ready_mutex);
```

Modifica la implementación de la cola de spool para que incluya todos estos cambios. Incluye esos cambios también en la gestión del acceso de los servidores de impresión.

2.5. Trabajo a presentar

Para la evaluación de la asignatura deberás presentar los resultados de esta práctica en la forma que te indique el profesor.

2.6. Apéndice 1

tst.c:

```
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <pthread.h>
#include "spool.h"

#define NJOBS           16
#define PROD_DELAY      10000
#define CONS_DELAY      50000

void* prod(void* data);
void* cons(void* data);

int main(int argc,char** argv)
{
        spool_t*        sp;
        pthread_t       prodid,consid;

        srand48(time(0));
        sp = spool_new();

        pthread_create(&prodid,0,prod,(void*)sp);
        pthread_create(&consid,0,cons,(void*)sp);

        pthread_join(prodid,0);
        pthread_join(consid,0);

        spool_delete(sp);

        return 0;
}

void* prod(void* data)
{
        spool_t* s = (spool_t*)data;
        job_t   j;
        int             i;

        for (i=0; i<NJOBS; i++)
        {
                usleep(drand48()*PROD_DELAY);
                j.owner = 0;
                j.id = i;
                spool_put(s,j);
                printf("lp %d %d\n",j.owner,j.id);
                fflush(stdout);
        }
        j.id = -1;
        spool_put(s,j);

        return 0;
}

void* cons(void* data)
{
        spool_t* s = (spool_t*)data;
        job_t   j;

        do
        {
                usleep(drand48()*CONS_DELAY);
                spool_get(s,&j);
                printf("done %d %d\n",j.owner,j.id);
                fflush(stdout);
        }
        while (j.id>=0);

        return 0;
}
```

spool.h:

```
#ifndef __SPOOL_H__
#define __SPOOL_H__

#ifndef __cplusplus
typedef struct job_t     job_t;
typedef struct spool_t   spool_t;
#endif

#define SPOOL_MAX        4

struct job_t
{
        int      owner;
        int      id;
};

struct spool_t
{
        job_t    jobs[SPOOL_MAX];
        int              in;
        int              out;
        int              slots;
        int              items;
};

spool_t* spool_new();
void spool_delete(spool_t* s);

int spool_put(spool_t* s,job_t job);
int spool_get(spool_t* s,job_t* job);

#endif /* __SPOOL_H__ */
```

spool.c:

```c
#include "spool.h"
#include <stdio.h>

spool_t* spool_new()
{
        spool_t* s;
        int             i;

        s = (spool_t*)malloc(sizeof(spool_t));
        s->in = 0;
        s->out = 0;
        s->slots = SPOOL_MAX;
        s->items = 0;

        return s;
}

void spool_delete(spool_t *s)
{
        free(s);
}

int spool_put(spool_t* s,job_t job)
{
        if (s->slots <= 0)
        {
                fprintf(stderr,"Error: spool queue full");
                abort();
        }
        s->jobs[s->in] = job;
        s->in++;
        s->in %= SPOOL_MAX;
        s->slots--;
        s->items++;
}

int spool_get(spool_t* s,job_t* job)
{
        if (s->items<=0)
        {
                fprintf(stderr,"Error: spool queue emtpy");
                abort();
        }
        *job = s->jobs[s->out];
        s->out++;
        s->out %= SPOOL_MAX;
        s->slots++;
        s->items--;
}
```

Práctica 3

Problema Tipo: Producto de Matrices Grafos

En esta práctica se propone la utilización de *threads* para la reolución de ciertos problemas de grafos mediante productos de matrices.

3.1. Introducción

Un grafo se puede representar en un computador mediante una *matriz de adyacencia* A, que simplemente contiene un valor booleano en cada casilla indicando si hay una conexión directa del nodo i (fila) al j (columna). Si el grafo es dirigido, la matriz no es simétrica.

Otra matriz importante que da más información es la *matriz de conectividad parcial* B_n, que indica si existe un camino de longitud menor o igual que n entre un par de nodos. Puede obtenerse mediante

$$B_1 = A + I(unosenladiagonal)$$

$$B_2 = B_1 * B_1 = B^2$$

$$B_3 = B_1 * B_1 * B_1 = B^3$$

$$\ldots$$

Como los elementos de las matrices son datos booleanos, las operaciones de suma se convierten el operaciones lógicas OR, y los productos de elementos en AND.

Como un camino no puede tener mayor longitud que el número de nodos menos 1, la *matriz de conectividad* C propiamente dicha se puede obtener como $C = B_{n-1}$.

Se quiere implementar la obtención de la matriz C mediante hilos POSIX. Para ello se implementará una función de producto de matrices quee trabaje en paralelo.

3.2. Balance de carga

La forma más simple de efectuar el proceso en paralelo consiste en hacer que cada thread calcule un conjunto de filas de la matriz producto.

Ello implica que tenemos que repartir el conjunto de filas entre los distintos hilos. Si esa decisión se toma al principio del programa, el reparto de carga puede no ser uniforme entre los hilos, o aunque lo sea, la distinta carga de los procesaroes puede hacer que algunos hilos se retrasen demasiado, mientras otros estan parados.

Para evitar eso, se pretende hacer un reparto dinámico de carga de la siguiente forma:

- Cada hilo calcula la fila que le indique una función de asignación de tareas, hasta que esa función indique que no queda más trabajo (por ejemplo, devuelviendo un valor mayor que el número de filas).
- Dicha función mantiene un contador con la siguiente tarea a asignar que se va incrementando. Debe controlar que dos hilos no puedan acceder a ella de forma concurrente, mediante algún tipo de bloqueo (mutex).

Implementa una funcion 'producto de matrices en paralelo' que debe crear los hilos, inicializar el asignador de tareas y esperar/destruir los hilos.

Mediante esa función, implementa la obtención de la matriz de conectividad de un grafo a partir de su matriz de adyacencia.

Parte II

Memoria Distribuida

Práctica 4

Introducción a MPI

En esta práctica se van a presentar las herramientas básicas de que se dispone en MPI para la construcción de programas paralelos en sistemas de memoria distribuida.

El sistema de prácticas está formado por los computadores del laboratorio, con los que se definirá una máquina paralela. El sistema utilizado es Linux, en particular la distribución CentOS 5.4, y la implementación de MPI es OpenMPI 1.3.2.

4.1. Configuración de OpenMPI

El sistema CentOS soporta la exsitencia de disintas implementaciones de MPI y que el usuario seleccione la que desea utilizar.

Además, para cada implementación de MPI se puede configurar su funcionamiento de distinta forma.

4.1.1. Selección de la implementacion de MPI

En CentOS, ejecuta el comando `mpi-selector-menu`:

```
lab004-073:~> mpi-selector-menu
Current system default: <none>
Current user default:   openmpi-1.3.2-gcc-i386

    "u" and "s" modifiers can be added to numeric and "U"
    commands to specify "user" or "system-wide".

1. openmpi-1.3.2-gcc-i386
U. Unset default
Q. Quit

Selection (1-1[us], U[us], Q):
```

Responde seleccionando la opcion '1', a nivel de usuario con 'u' y sal con 'q':

```
Selection (1-1[us], U[us], Q): 1
Operator on the per-user or system-wide default (u/s)? u
Defaults already exist; overwrite them? (y/N) y

Current system default: <none>
Current user default:   openmpi-1.3.2-gcc-i386

    "u" and "s" modifiers can be added to numeric and "U"
    commands to specify "user" or "system-wide".

1. openmpi-1.3.2-gcc-i386
U. Unset default
Q. Quit

Selection (1-1[us], U[us], Q): q

WARNING: Changes made to mpi-selector defaults will not be visible until
you start a new shell!
```

Sal de la cuenta y vuelve a entrar. Si tienes los ficheros de configuración por defecto, prueba a ejecutar el comando `mpirun -V`:

```
lab004-073:~> mpirun -V
mpirun (Open MPI) 1.3.2

Report bugs to http://www.open-mpi.org/community/help/
```

Si no funciona, es porque has modificado la configuración de tu `.profile`. Editalo y añade la siguiente linea:

```
source /etc/profile.d/mpi-selector.sh
```

Sal y vuelve a entrar en la cuenta y prueba `mpirun -V`.

4.1.2. Configuración de OpenMPI

OpenMPI puede usar distintas posibilidades para ciertas tareas, en particular:

- Ejecución remota
- Interfaces de red

Para configurarlas en CentOS, debes crear un directorio en tu `HOME` llamado `.openmpi`, y dentro de el un fichero llamado `mca-params.conf` con el siguiente contenido:

```
# Protocolos de ejecucion remota
btl=tcp,sm,self
# Metodos de ejecucion remota
plm_rsh_agent=ssh
# Interfaz de red
btl_tcp_if_include=br0
```

4.1.3. Configuración de OpenSSH

Para que sl comando ssh permita la ejecución remota de comandos de MPI (y las conexiones remotas) sin preguntar password, es necesario configurar tu pareja de claves publica/privada.

Para eso, ejecuta lo siguiente:

```
lab004-073:~/> ssh-keygen -t dsa
....(contesta a todo con <enter>)...
lab004-073:~/> cd .ssh
lab004-073:~/.ssh> cat id_dsa.pub >> authorized_keys
```

Prueba a ejecutar un comando remoto:

```
lab004-073:~> ssh localhost pwd
...
lab004-073:~> ssh lab004-073 pwd
...
```

4.2. Ejecución de programas en MPI

Puedes descargar el programa de ejemplo de la sección de Prácticas de la página de la asignatura

`http://webdiis.unizar.es/asignaturas/PPAR/prc/`

Para compilarlo teclea simplemente `make`.

Un programa MPI se puede ejecutar como un programa normal:

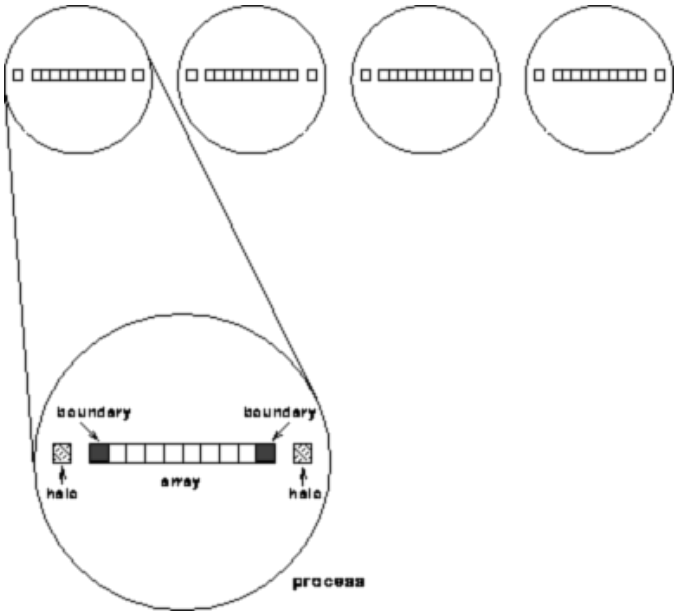
```
lab004-073:~/dev/mpi/ej0> tst
Soy lab004-073, 0 de 1 nodos
Time:         0.00 ms
Bandwidth: 63717641.85 Mb/s
```

Para ejecutar un programa en MPI sobre una maquina paralela formada por varias maquinas independientes se usa el comando `mpirun`. Ese comando necesita un fichero de configuración donde se listan las maquinas a usar:

```
lab004-073:~/dev/mpi/ej0> cat hf
lab004-071
lab004-072
lab004-073
lab004-074
lab004-075
lab004-073:~/dev/mpi/ej0> mpirun --hostfile hf tst
Soy lab004-073, 2 de 5 nodos
Soy lab004-071, 0 de 5 nodos
Soy lab004-072, 1 de 5 nodos
Soy lab004-075, 4 de 5 nodos
Soy lab004-074, 3 de 5 nodos
Time:      1568.34 ms
Bandwidth:   81.62 Mb/s
```

4.3. Trabajo práctico

Escribe un programa que realice un suavizado unidimensional sobre un vector. El vector estará repartido entre los diversos procesos que formen el programa paralelo (memoria distribuida):



El algoritmo debe funcionar de forma iterativa, en 3 fases:

- Inicialización y reparto de datos:
  El proceso principal (rango 0) envía a cada uno de los demás su correspondiente trozo de vector.

- Iteración:
  Cada uno de los procesos ejecuta el suavizado del vector, generando un vector nuevo. Para ello necesita los valores del halo de sus vecinos. Se calcula un valor de error global como el modulo de la diferencia entre los dos vectores. Ese error global debe ser conocido por todos los procesos. Se repiten los dos pasos anteriores hasta que el error sea menor que una cota dada.

- Recogida de resultados y finalización
  El programa principal recoge los resultados.

Implementa ese programa sobre MPI.

Práctica 5

Resolución iterativa de sistemas lineales

En esta práctica se va utilizar MPI para resolver de forma iterativa un sistema de ecuaciones lineales. La implementacion sobre MPI permitira resolver sistemas más grandes que lo que sería posible con una única máquina.

5.1. Resolución iterativa de sistemas de ecuaciones lineales

Un sistema de ecuaciones lineales se define mediante la ecuación

$$A \cdot x = b$$

donde A es una matriz de dimensiones $N \times N$, y x y b son vectores columna de longitud N.

Un sistema de ecuaciones puede resolverse por un método iterativo denominado método de Jacobi (puedes ver una descripción más detallada del método en http://en.wikipedia.org/wiki/Jacobi_method). Dicho método parte de una solución inicial aproximada $x^{(0)}$ y va generando de forma iterativa nuevas versiones de la solución $x^{(k+1)}$ más cercanas a la final a partir de una $x^{(k)}$:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)})$$

En cada iteración se genera una nueva solución $x^{(k+1)}$ a partir de $x^{(k)}$. El proceso se repite hasta que el error baja de una cota mínima. El error puede medirse como la diferencia máxima entre las componentes de los vectores $x^{(k)}$ y $x^{(k+1)}$.

EL programa de partida de la práctica implementa ese método de forma secuencial.

5.2. Implementación sobre MPI

El objetivo de la práctica es distribuir entre las máquinas del cluster tanto la matrix A, como los vectores b, $x^{(k)}$ y $x^{(k+1)}$. Cada nodo sólo almacenará una banda de la matriz, y un trozo de los vectores correspondientes.

En cada paso, cada nodo debe conseguir la información necesaria de los demás para implementar los cálculos de la iteración, y calcular sus nuevos valores locales para $x^{(k+1)}$.

Para terminar el algoritmo, en cada paso debe implementarse el cálculo del error global mediante un proceso de reducción.

5.3. Resultados

El resultado de la práctica debe ser, además del propio programa, un documento de texto en el que se refleje información sobre:

- El diseño del algoritmo paralelo
- Su implementación
- Las prestaciones obtenidas, tanto a nivel de tiempos de cálculo (comparando con el algoritmo secuencial), como a nivel de volumen de datos tratados (máximo tamaño de matriz con el que se ha podido trabajar).

5.4. Trabajo adicional

Una vez que se tiene el algoritmo de memoria distribuida implementado con MPI, dentro de cada uno de los nodos los cáclulos pueden a su vez paralelizarse mediante hilos POSIX.

Implementa la paralelización local con hilos y evalúa también sus prestaciones.